Arboles

Un árbol, es una estructura jerárquica conformada por nodos relacionados entre sí, donde:

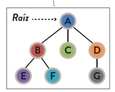
A.
Cada nodo puede tener uno o más nodos que cumplen el rol de hijos

B.
Todo nodo, excepto la raíz, tiene un nodo que cumple el rol de padre

C.
El nodo raíz no tiene padre.

Sirven para almacenar los datos de forma jerarquica

- Organigramas
- Sistema de archivos de carpetas



Arbol binario

Un árbol binario, es una estructura de datos recursiva que está compuesta por un valor, llamado raíz, y por dos árboles, llamados subárbol izquierdo y subárbol derecho. Formalmente:

Un árbol binario, es un árbol donde todo nodo tiene máximo dos hijos: el hijo izquierdo y el hijo derecho.

Tabla 1. Ejemplos de árboles binarios

Árbol binario	Raíz	Subárbol izquierdo	Subárbol derecho
	no tiene	no tiene	no tiene
	30	*	*
	87	*	
	57		

Fuente: Satudo (s.f.)

Nótese que:

•El árbol vacío no tiene raíz, no tiene subárbol izquierdo y no tiene subárbol derecho.

•Todo árbol no vacío tiene raíz y tiene exactamente dos subárboles: el subárbol izquierdo y el subárbol derecho. Es posible que uno o ambos subárboles sean vacíos.

•El árbol vacío nunca puede ser considerado como nodo, porque no es raíz de un árbol binario no vacío.

Nodos

En los tres últimos ejemplos de la tabla, 57, 11, 87, 80, 74 y 30 son los nodos que conforman los árboles.

Hojas y nodos internos

Un nodo es una hoja si sus dos subárboles son vacíos (no tiene hijos).

y es un nodo interno si no es una hoja (tiene al menos un hijo).

Árbol binario	Ejemplos
	El número total de nodos de árboles es 10. La raíz del árbol es el nodo con valor 21. Los nodos 80, 74, 87, 89 y 10 son hojas porque no tienen hijos. Los nodos 21, 57, 98, 11 y 23 son nodos internos porque no son hojas. Todo nodo del árbol es una hoja o es un nodo interno.

Conceptos básicos

- Relaciones de parentesco
- Descendientes y ancestros
- Caminos y ramas
- Nivel
- Peso y altura

Relaciones de parentesco

Tabla 2. Relaciones de parentesco entre nodos

Parentesco	Definición			
Hijo	Un nodo f	es hijo de	un nodo g	si y sólo si f es la raíz de alguno de los subárboles de g.
Padre	Un nodo f	es padre de	un nodo g	si y sólo si g es hijo de f.
Hermano	Un nodo f	es hermano de	un nodo g	si y sólo si f y g son distintos y tienen el mismo padre.
Nieto	Un nodo f	es nieto de	un nodo g	si y sólo si f es hijo de un hijo de g.
Abuelo	Un nodo f	es abuelo de	un nodo g	si y sólo si g es nieto de f.
Tio	Un nodo f	es tío de	un nodo g	si y sólo si f es hermano del padre de g.
Sobrino	Un nodo f	es sobrino de	un nodo g	si y sólo si g es tío de f
Primo	Un nodo f	es primo de	un nodo g	si y sólo si el padre de f es hermano del padre de g.

Descendientes y ancestros

Un nodo f es ancestro de un nodo g si y solo si hay un camino que va de f a g.

Árbol binario	Nodo	Ancestros	Descendientes
	57	57	57, 11, 87, 80 y 74
	11	11 y 57	11, 80 y 74
	87	87 y 57	87
	80	80, 11 y 57	80
	74	74, 11 y 57	74

Caminos y ramas

Un camino es una lista no vacía de nodos donde cada uno es padre del que le sigue en la lista. La longitud de un camino es n - 1, o sea, el número de nodos que enumera menos uno.

Además, se define una rama como un camino que parte de la raíz del árbol y llega a una hoja.

Árbol binario	Ejemplos
	Hay 6 caminos de longitud 0: (57), (11), (87), (80). (74), y (30). Hay 5 caminos de longitud 1: (57, 11), (57,87), (11,80), (11, 74). y (87, 30). Hay 3 caminos de longitud 2: (57, 11, 80), (57, 11, 74), y (57,87,30). No hay caminos de longitud 3 en adelante. 80, 74 y 30 son hojas. Las ramas son (57, 11, 80), (57, 11, 74) (57,87, 30).
	(87,57) no es camino porque 87 no es padre de 57. (80,11,57,87) no es camino pues 11 no es padre de 57 y 80 no es padre de 11. (80, 87) tampoco es camino. NO hay camino que vaya desde 80 hasta 87. Hay camino del nodo 57 hacia todos los demás porque 57 es la raíz del árbol.

Siempre existe un camino de longitud cero que va desde un nodo hacia sí mismo.

siempre existe un camino que va desde la raíz del árbol hasta cualquier nodo

Nivel

El nivel de un nodo es la longitud del camino que va desde la raíz hasta el mismo nodo.

Se dice que un nivel está lleno si alberga la máxima cantidad posible de nodos que puede tener.

Árbol binario	Ejemplos
	El único nodo que está en el nivel 0 es el 21. Los nodos que están en el nivel 1 son 57 y 98. Los nodos que están en el nivel 2 son 11, 87, 23 y 10. Los nodos que están en el nivel 3 son 80, 74 y 89. No hay nodos que estén desde el nivel 4 en adelante. Los niveles 0, 1 y 2 están llenos. El nivel 3 no está lleno.

el nivel k puede tener máximo 2k nodos porque cada nivel alega máximo el doble de los nodos que el nivel anterior.

Peso y altura

La altura (n) de un árbol binario es uno más la longitud de la rama más larga

El peso (n) de un árbol binario es el número de nodos que tiene.

Árbol binario	Longitud de rama más larga	Peso (n)	Altura (h)
	No tiene ramas.	0	0
	0	1	1
	2	5	3
	3	10	14

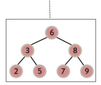
Tipos de arboles binarios

- Árboles degenerados — Un árbol es degenerado cuando su altura es igual a su peso.
- Arboles Llenos — Un árbol está lleno cuando todos sus nodos tienen exactamente cero o dos hijos
- Arboles completos — Un árbol está completo cuando todos sus niveles hasta el penúltimo están llenos, y todos los nodos del último nivel están lo más a la izquierda posible.
- Arboles perfectos — Un árbol es perfecto cuando todos sus niveles están llenos.

Árbol binario	¿Degenerado?	¿Lleno?	¿Completo?	¿Perfecto?
	SI	SI	SI	SI
	SI	NO	NO	NO
	NO	SI	NO	NO
	NO	NO	SI	NO
	NO	SI	SI	NO
	NO	SI	SI	SI

Arboles binarios ordenados

Por otro lado, un árbol ordenado es un árbol binario donde cada nodo cumple que su valor es mayor que todos los valores a su izquierda, y que es menor que todos los valores a su derecha



Tipos Comunes de Árboles:

1. Árbol Binario
Cada nodo tiene como máximo dos hijos: un hijo izquierdo y un hijo derecho.

2. Árbol Binario de Búsqueda (BST)
Un tipo especial de árbol binario donde para cada nodo:
Los valores en el subárbol izquierdo son menores que el valor del nodo.
Los valores en el subárbol derecho son mayores que el valor del nodo.
Este tipo de árbol permite búsquedas eficientes.

3. Árbol AVL
Un tipo de **árbol binario de búsqueda** que se mantiene balanceado automáticamente.
La diferencia de altura entre los subárboles izquierdo y derecho de cualquier nodo es, como máximo, 1.

4. Árbol Rojo-Negro
Otro tipo de **árbol binario balanceado** que garantiza que la altura del árbol sea aproximadamente logarítmica en el número de nodos.
Es útil para operaciones rápidas de búsqueda, inserción y eliminación.

5. Heap o Montículo
Un **árbol binario** completo donde se satisface la propiedad del **montículo** (heap):
En un **min-heap**, el valor de un nodo es menor que los valores de sus hijos.
En un **max-heap**, el valor de un nodo es mayor que los valores de sus hijos.

Diferencias entre Árbol Binario Común y Árbol Binario Ordenado:

Árbol Binario Común: No sigue ninguna regla estricta en cuanto al valor de los nodos. Los nodos pueden colocarse en cualquier lugar sin restricciones, lo que hace que las búsquedas sean menos eficientes.

Árbol Binario Ordenado (BST): Sigue una propiedad específica de orden, lo que permite realizar búsquedas, inserciones y eliminaciones de manera más rápida y eficiente

Árbol Binario de Búsqueda (BST)

Un tipo especial de árbol binario donde para cada nodo:

Los valores en el subárbol izquierdo son menores que el valor del nodo.
Los valores en el subárbol derecho son mayores que el valor del nodo.

Este tipo de árbol permite búsquedas eficientes.

Implementacion en java

```
// Definición de la clase Nodo
class Nodo {
    int valor;
    Nodo izquierdo, derecho;

    public Nodo(int valor) {
        this.valor = valor;
        izquierdo = derecho = null;
    }
}

// Definición de la clase Árbol Binario de Búsqueda (BST)
class ArbolBinarioOrdenado {
    Nodo raiz;

    public ArbolBinarioOrdenado() {
        raiz = null;
    }

    // Método para insertar un nuevo nodo en el árbol
    public void insertar(int valor) {
        raiz = insertarRecursivo(raiz, valor);
    }

    // Método recursivo para insertar un nuevo valor en el árbol
    private Nodo insertarRecursivo(Nodo raiz, int valor) {
        // Si el árbol está vacío, se crea un nuevo nodo
        if (raiz == null) {
            raiz = new Nodo(valor);
            return raiz;
        }

        // Si el valor es menor que el nodo actual, se inserta en el subárbol izquierdo
        if (valor < raiz.valor) {
            raiz.izquierdo = insertarRecursivo(raiz.izquierdo, valor);
        }
        // Si el valor es mayor que el nodo actual, se inserta en el subárbol derecho
        else if (valor > raiz.valor) {
            raiz.derecho = insertarRecursivo(raiz.derecho, valor);
        }

        return raiz;
    }

    // Método para recorrer el árbol en orden (in-order traversal)
    public void recorridoEnOrden() {
        recorridoEnOrdenRecursivo(raiz);
    }

    // Método recursivo para el recorrido en orden
    private void recorridoEnOrdenRecursivo(Nodo raiz) {
        if (raiz != null) {
            recorridoEnOrdenRecursivo(raiz.izquierdo);
            System.out.print(raiz.valor + " ");
            recorridoEnOrdenRecursivo(raiz.derecho);
        }
    }

    // Método principal para probar la implementación
    public static void main(String[] args) {
        ArbolBinarioOrdenado arbol = new ArbolBinarioOrdenado();

        // Insertamos valores en el árbol
        arbol.insertar(50);
        arbol.insertar(30);
        arbol.insertar(20);
        arbol.insertar(40);
        arbol.insertar(70);
        arbol.insertar(60);
        arbol.insertar(80);

        // Imprimimos el recorrido en orden
        System.out.print("Recorrido en orden: ");
        arbol.recorridoEnOrden(); // Salida esperada: 20 30 40 50 60 70 80
    }
}
```